# Module #11: Imputation Methods

In this first section of the class, we demonstrate the methods built into the R packages `mice()` and `VIM()` to visualize missingness patterns and conduct single imputation for one variable at a time. We will not walk through an example for all available methods available, of which there are several dozen, but rather a select few. One can consult the documentation for additional capabilities.

In the second part of the class, we introduce the idea of multiple imputation, illustrating how to create multiple completed data sets and combine inferences from each into a single analysis.

# Unit Nonresponse versus Item Nonresponse

*Unit nonresponse* occurs when all survey items are missing.

*Item nonresponse* occurs when some, but not all, survey items are missing.

Imputation can be used for either type, but is most often used to handle item nonresponse – a survey data set could require both.

**Unit Nonresponse**

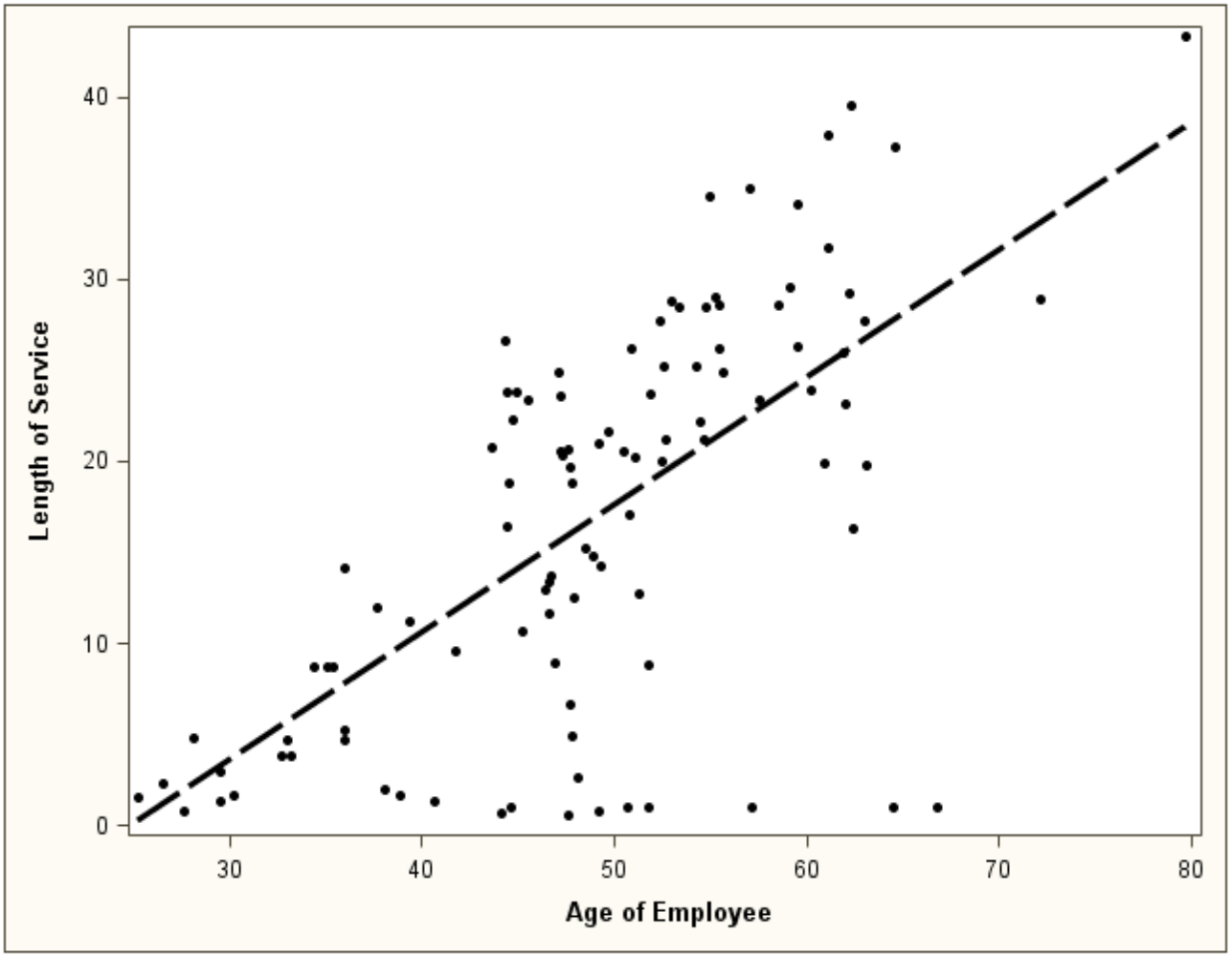
| | Outcome Variables | | | |
|---|---|---|---|---|
| **X** | $y_1$ | $y_2$ | $y_3$ | $y_4$ |
| | | | | |
| | ? | ? | ? | ? |
| | | | | |
| | ? | ? | ? | ? |
| | | | | |

**Item Nonresponse**

| | Outcome Variables | | | |
|---|---|---|---|---|
| **X** | $y_1$ | $y_2$ | $y_3$ | $y_4$ |
| | | | ? | |
| | | ? | | |
| | ? | | | |
| | | | | ? |

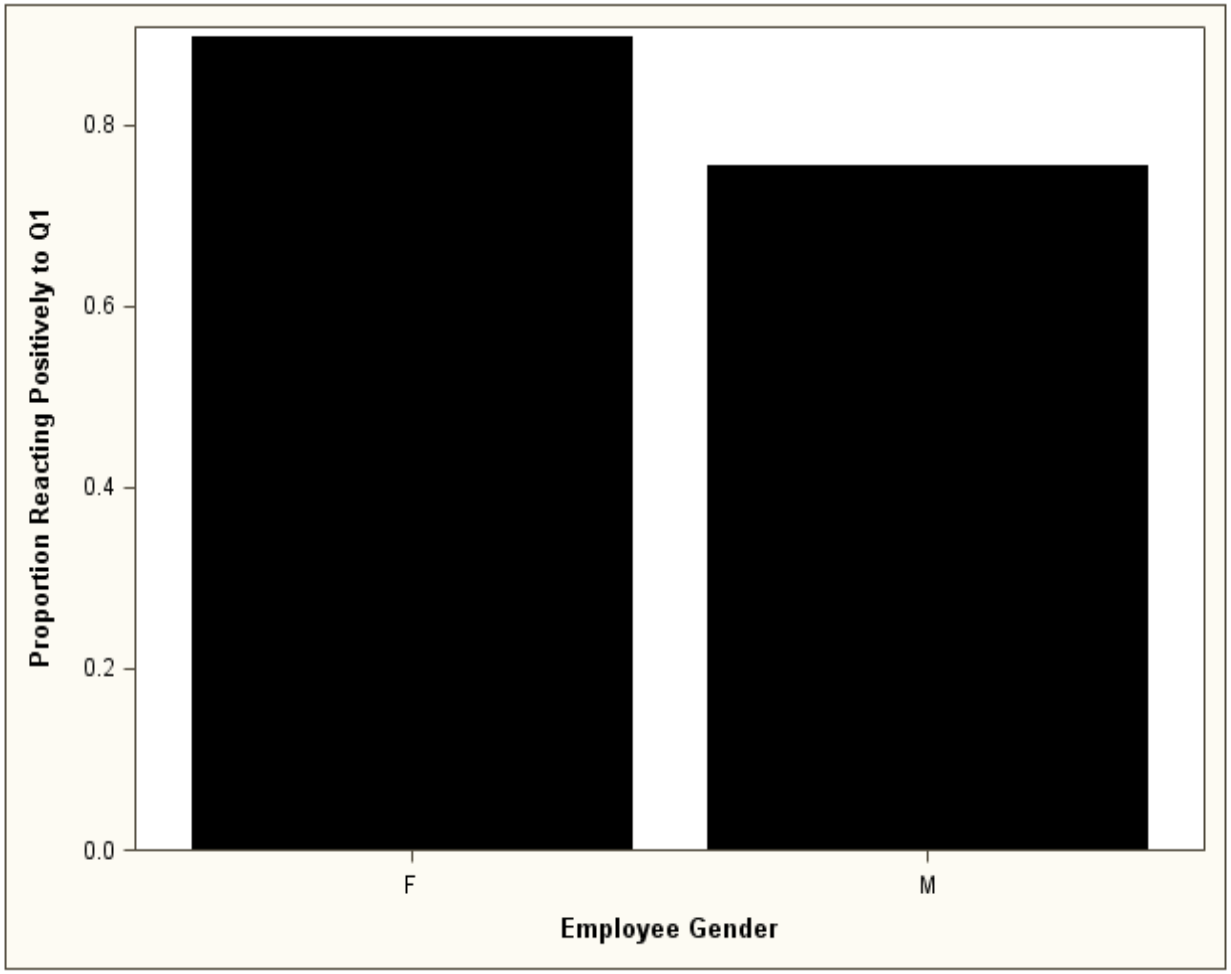
# Visualizing Imputation with a Continuous Auxiliary Variable

Suppose we wanted to impute length of service. Doesn't an employee's age tell us *something* about his/her length of service?



# Visualizing Imputation with a Categorical Auxiliary Variable

Suppose we wanted to impute the 0/1 indicator of a positive response to Q1. Doesn't knowing an employee's gender tell us *something* about the expected value of Q1?



# Unconditional Mean Imputation

A naïve, but very common, approach is to fill in the *m* missing values with the overall mean of the *r* observed values (*r* + *m* = *n*). This assumes data are missing completely at random (MCAR), but even when that is true, imputing the overall mean results in a potentially severe underestimation of variance.

For example, consider the variance of a sample mean:

$$\operatorname{var}(\hat{\bar{y}}) = \frac{s^2}{r} = \frac{\sum_{i \in r} \left( y_i - \hat{\bar{y}}_r \right)^2}{r(r-1)}$$

Replacing the missing values with the mean retains the same sample mean and summation term, but *n* (> *r*) replaces *r* in the denominator, so the variance is artificially reduced. This is unjustified because we added no new information.

# Deterministic vs. Stochastic Imputation

Imputing the mean is an example of a *deterministic imputation model*:

$$\text{for } i \in m,\ y_{\text{i}} = \hat{\bar{y}}_r$$

An example of a *stochastic imputation model* is adding a random residual term $\varepsilon_i$

$$\text{for } i \in m,\ y_{\text{i}} = \hat{\bar{y}}_r + \varepsilon_i$$

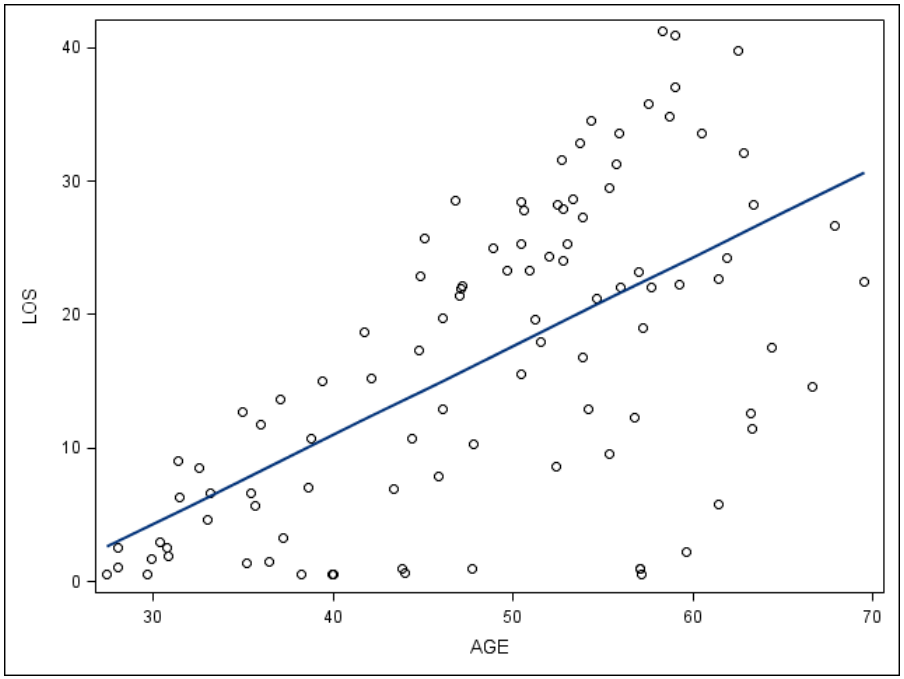
# Advantages and Disadvantages of Stochastic Imputation Models

The biggest benefit of a stochastic imputation model relative to a deterministic model is less severe underestimation of variance. And in the case of dichotomous outcome variable like Q1, it would not make sense to impute the same value (either 0 or 1) or a fixed proportion for all missing cases. There are similar limitations when imputing the mode or median.

One potential headache when utilizing a stochastic model is convincing ourselves about the legitimacy of the assumed residual term distribution:

- Many models assume residuals have constant variance
- We could have nonsensical values returned

# Imputing LOS Based on Employee Age: Illustration of Stochastic vs. Deterministic Model




$$LOS_i = \beta_0 + \beta_1(AGE_i) + \varepsilon_i$$

$$\hat{\beta}_0 = -15.66$$

$$\hat{\beta}_1 = 0.67$$

$$\text{var}(\varepsilon_i) = MSE = 90.98$$

Considering a nonrespondent who is 40 years old:

Deterministic: $LOS_i = -15.66 + 0.67(40) = 11.14$

Stochastic: $LOS_i = -15.66 + 0.67(40) + z_i\sqrt{90.98}$ $\quad z_i \sim N(0,1)$

# Explicit Imputation Models

The model of LOS based on age just shown is an example of an *explicit model*, one with "betas," model parameters that we estimate.

Depending on the variable type/scale, we can formulate a wide range of explicit models. The table below summarizes a few possibilities:

| Variable Type | Example | Model Type |
|---|---|---|
| Dichotomous | Male/Female | Logistic |
| Ordinal (Ordered Categorical) | Likert-Type Scale Coded 1, 2, …, 5 | Ordinal Logistic Regression |
| Nominal (Unordered Categorical) | Work Occupation | Multinomial Logistic Regression |
| Count | Number of Doctor Visits in a Year | Poisson Regression |
| Semi-Continuous | Number of Years as a Smoker | Combination of Logistic and Linear Regression |

# Implicit Imputation Models

Alternatively, we could formulate an *implicit model* by matching *donors* and *beggars* within cells – this is termed *hot-deck* imputation.

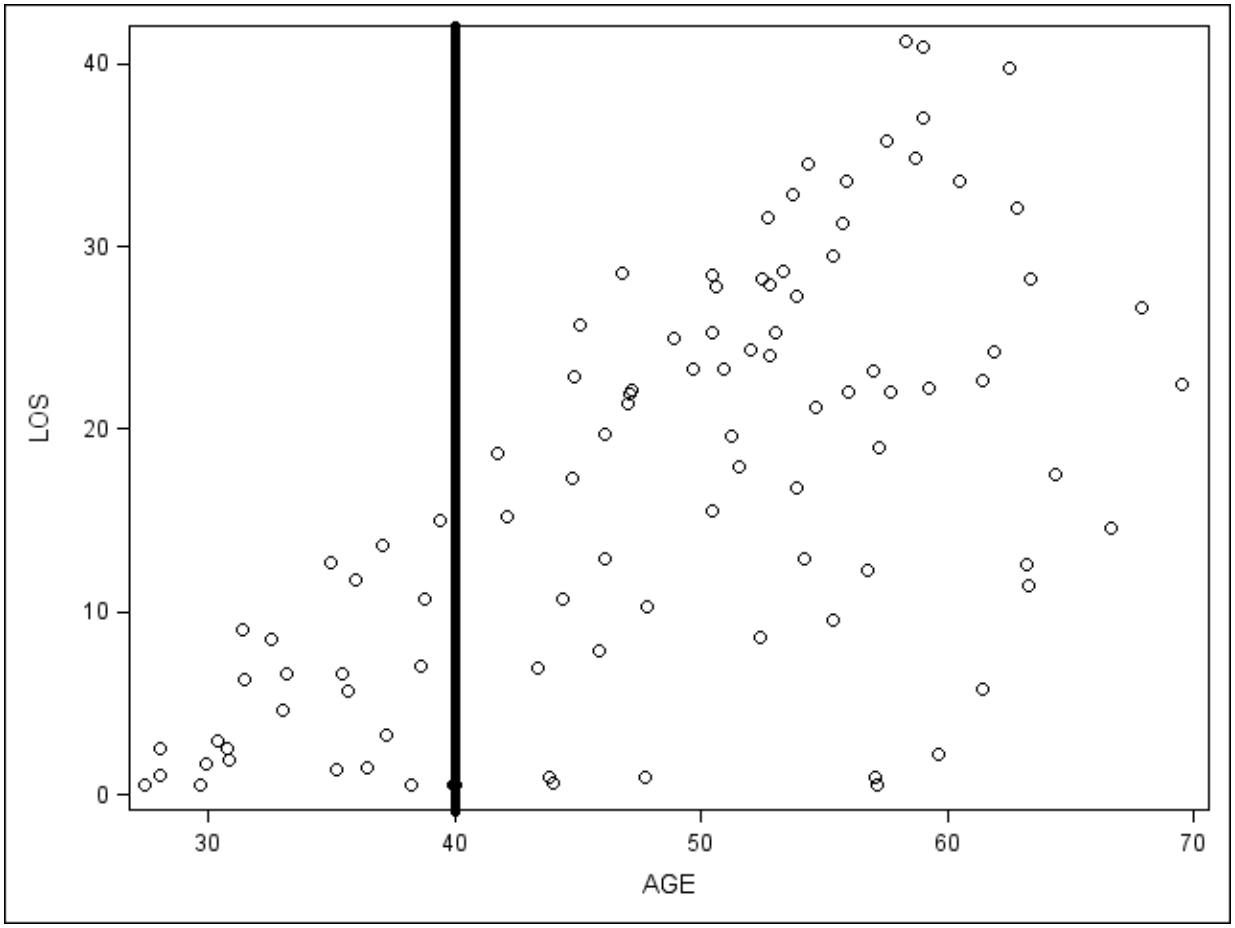
The notion of hot-deck imputation is to fill in the $m_c$ missing values within cell $c$ by selecting a random sample (with replacement) of $m_c$ values from $r_c$, the set of non-missing values with the cell. In the implicit model case, you can think of model terms (betas) as indicator variables of cell membership.

From Andridge and Little (2010):

> Historically, the term "hot deck" comes from the use of computer punch cards for data storage, and refers to the deck of cards for donors available for a non-respondent. The deck was "hot" since it was currently being processed, as opposed to the "cold deck" which refers to using pre-processed data as the donors, i.e. data from a previous data collection or a different data set. (p. 41)

# Imputing LOS Based on Age: Illustration of an Implicit Model

Image below visualizes how we could partition the sample into two cells based on whether an employee is younger/older than 40 (using the AGECAT variable), and impute the missing values therein.



# Advantages and Disadvantages of Implicit (Cell-Based) Imputation Models

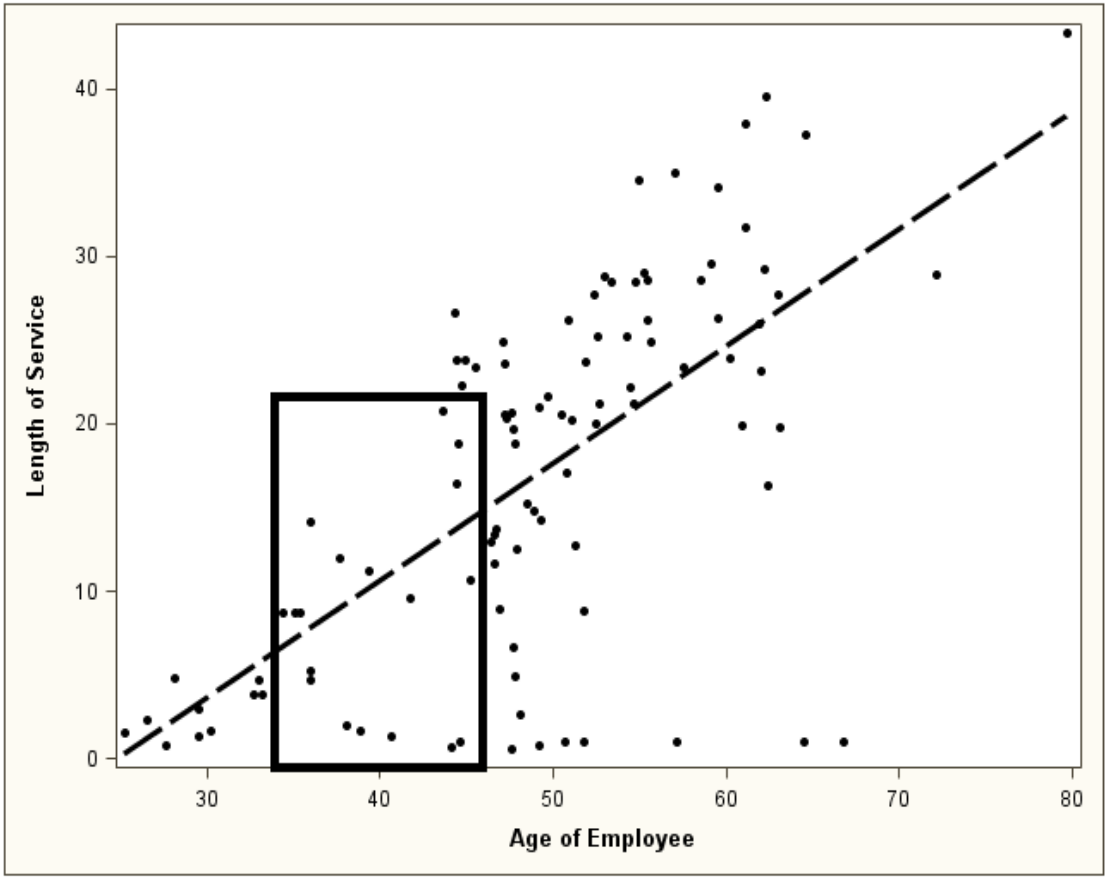
Imputed values are "real." For instance, we can rest assured knowing there will be no negative length of service values imputed.

There are no variable type requirements; that is, these methods are scale-indifferent and distribution free. This is also true of residuals (e.g., no assumptions of normality).

Similarly to the adjustment cell method for weighting, one notable downside is that forming cells based on more than a few covariates can lead to small cell sizes → generally speaking, you do not want a particular donor's value used "too frequently."

# Illustration of a Semi-Parametric Imputation Model

Technically, one could purse a *semi-parametric* imputation technique (Heitjan and Little, 1991; Schenker and Taylor, 1996), which have been devised to effectively bridge the gap between explicit and implicit models, drawing upon advantages of both. For example, we could fit a linear regression model but draw a residual from neighboring observed values.



# Variables on Data Set "Sample"

Design-Related Variables:

1. SUPERVISOR – 0/1 stratum indicator of being a supervisor
2. WEIGHT_BASE – the sampling weight, or inverse of selection probability

Additional Auxiliary Variables from Personnel Database, **X**:

1. GENDER – F/M indicator
2. MINORITY – Y/N indicator of being a minority race/ethnicity (i.e., non-white)
3. AGE – year/decimal employee age representation (e.g., 40.125)

Survey Variables, **Y**:

1. Q1_FULL – 1, 2, …, 5 response on a Likert-type response scale to the question "I recommend my organization as a good place to work," ordered form "Completely Disagree" to "Completely Agree"
2. Q1 – reduction of Q1_full to a 0/1 indicator of a positive response (i.e., "Completely Agree" or "Agree")
3. LOS – year/decimal representation of length of service with organization

# Tools for Visualizing the Missing Data Pattern

A basic function within the `mice()` package is `md.pattern()`. For the data set specified in parentheses, it will output a concise summary of the missingness pattern, both horizontally and vertically. The syntax below demonstrates:

```
# load R package
> library(mice)

# read in sample data
> sample <- read.table(file = "C:/Users/THLewis/Desktop/Course
            Materials/Data/sample.csv", sep=",", header=T)

# examine missingness pattern
> md.pattern(sample)
```

```
    GENDER Minority AGE agecat SUPERVISOR weight_base respond  Q1 Q1_full LOS
628      1        1   1      1          1           1       1   1       1   1   0
572      1        1   1      1          1           1       1   0       0   0   3
         0        0   0      0          0           0       0 572     572 572 1716
```

# Tools for Visualizing the Missing Data Pattern (2)

Another potentially useful function within the `mice()` package is `md.pairs()`. For each pair of variables in the data set specified in parentheses, four tables summarizing the counts of observed/missing combinations are output as follows:

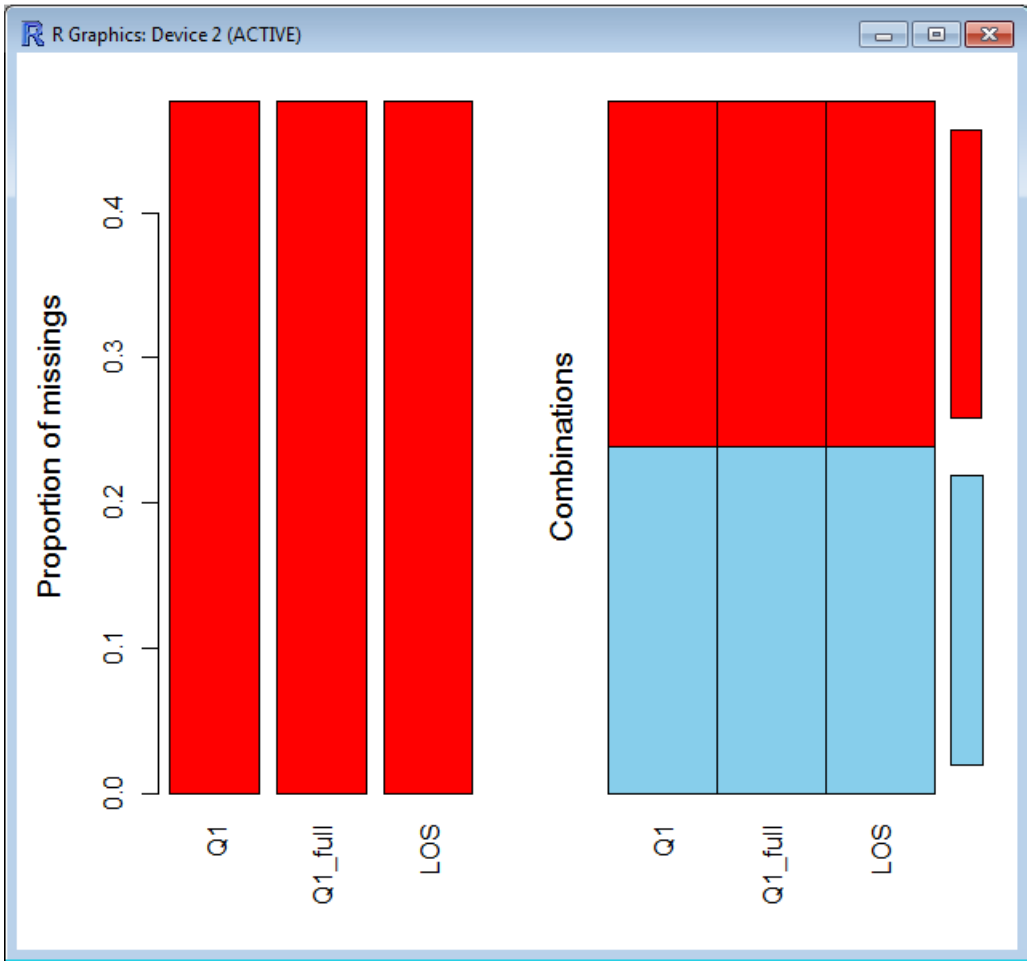
```
> surv.items <- sample[,8:10]
> md.pairs(surv.items)
```

```
$rr
          Q1 Q1_full LOS
Q1       628      628 628
Q1_full  628      628 628
LOS      628      628 628

$rm
         Q1 Q1_full LOS
Q1        0       0   0
Q1_full   0       0   0
LOS       0       0   0

$mr
         Q1 Q1_full LOS
Q1        0       0   0
Q1_full   0       0   0
LOS       0       0   0

$mm
          Q1 Q1_full LOS
Q1       572      572 572
Q1_full  572      572 572
LOS      572      572 572
```

# Tools for Visualizing the Missing Data Pattern (3)

Within the package `VIM()`, the `aggr()` function will output a bar chart of sorts to help visualize variable-specific missingness rates across the entire data set.

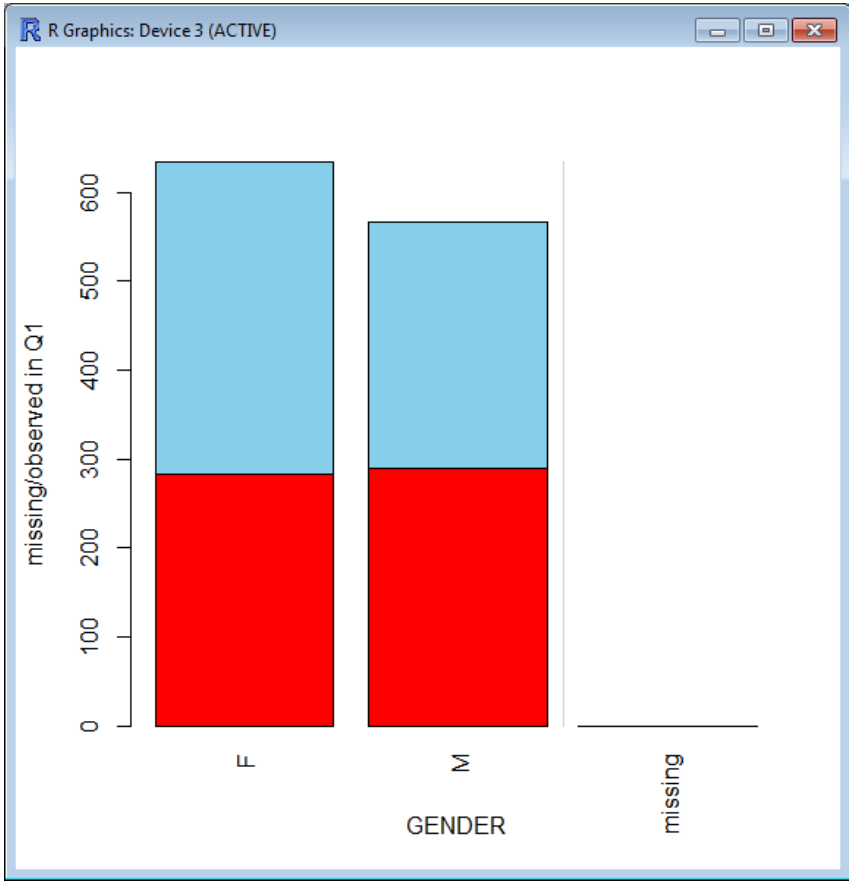
```
> library(VIM)
> aggr(surv.items)
```



# Tools for Visualizing the Missing Data Pattern (4)

Also within the package `VIM()`, the `barMiss()` function is potentially useful for comparing the distribution of missingness across a fully observed covariate.

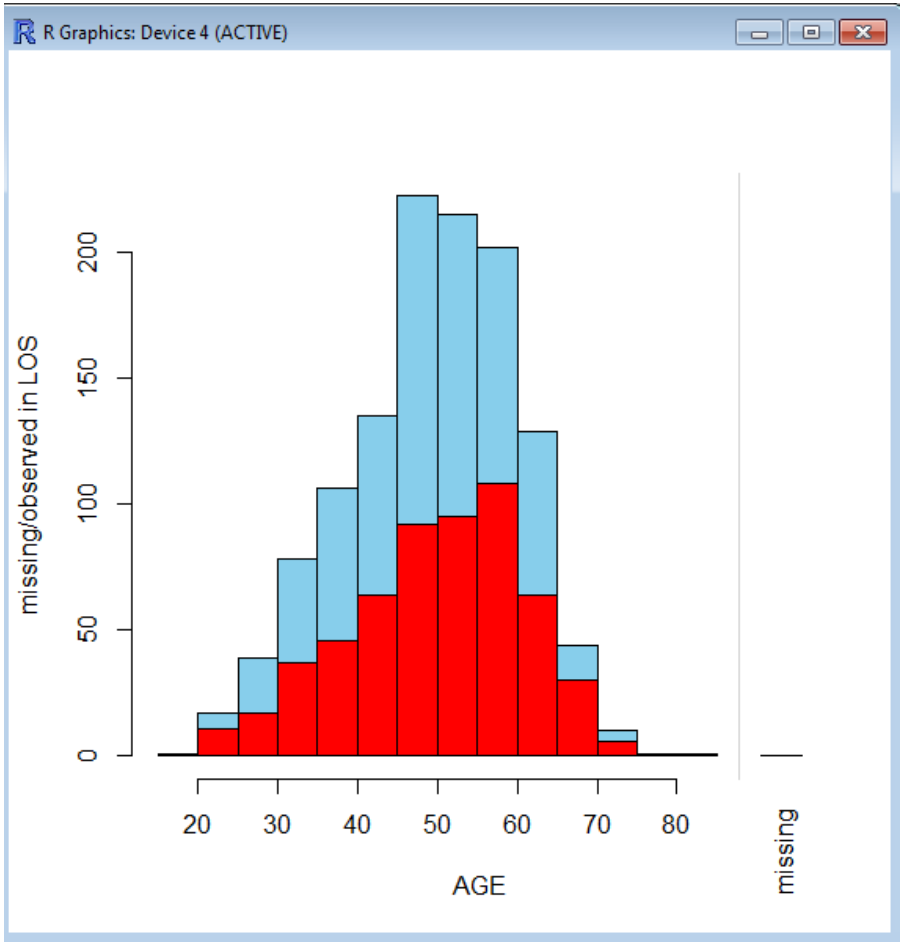
```
> barMiss(sample[,c("GENDER","Q1")])
```



# Tools for Visualizing the Missing Data Pattern (5)

Lastly, the `histMiss()` function within the `VIM()` package is the analog for when a fully observed covariate on a continuous scale.

```
> histMiss(sample[,c("AGE","LOS")])
```



# Unconditional Mean Imputation

Despite the perils of unconditional mean imputation, the reality is that it is still used frequently in practice. As such, note that you can conduct this method for a continuous variable if desired as follows:

```
# subset data to only columns needed for imputation
> tomice.LOS <-
      sample[,c("GENDER","Minority","AGE","SUPERVISOR","LOS")]

# example of mean imputation
> imp <- mice(tomice.LOS,method="mean",m=1)
# NOTE: we can modify the parameter m to a value greater than 1
for multiple imputation

# command below stores the result (i.e., a completed data set)
# in a separate object
> frommice.LOS <- complete(imp)
```

# Examining the Impact of Imputation

One common diagnostic tool to assess the impact of imputation is to contrast the distribution of observed cases to that of the completed data set, complete cases plus imputed values. Below is some basic syntax to do so—if the variable is categorical, one can swap the `table()` function for the `summary()` function.

```
> summary(tomice.LOS$LOS)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
 0.0975  5.6650 16.3300 16.2700 24.7500 43.3300     572
```

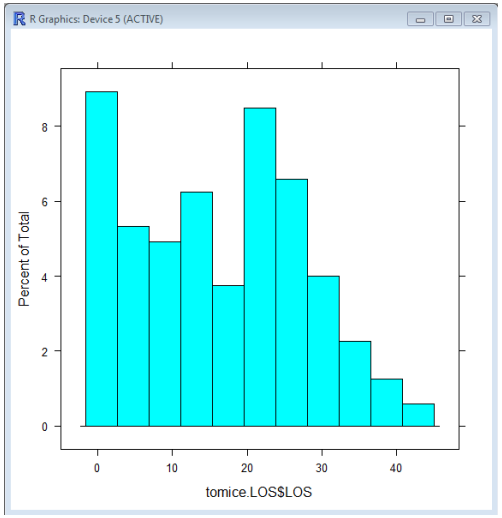
```
> summary(frommice.LOS$LOS)
```

```
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
 0.09748 15.17000 16.27000 16.27000 17.79000 43.33000
```

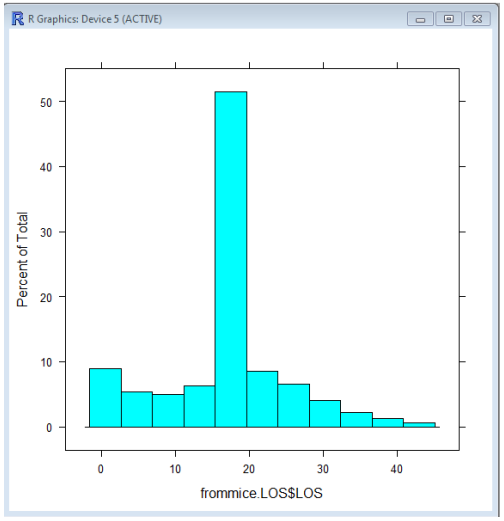
# Examining the Impact of Imputation (2)

Another handy visualization tool is the `histogram()` function. Notice how imputing the mean distorts the overall distribution of LOS:

```
> histogram(tomice.LOS$LOS)
```



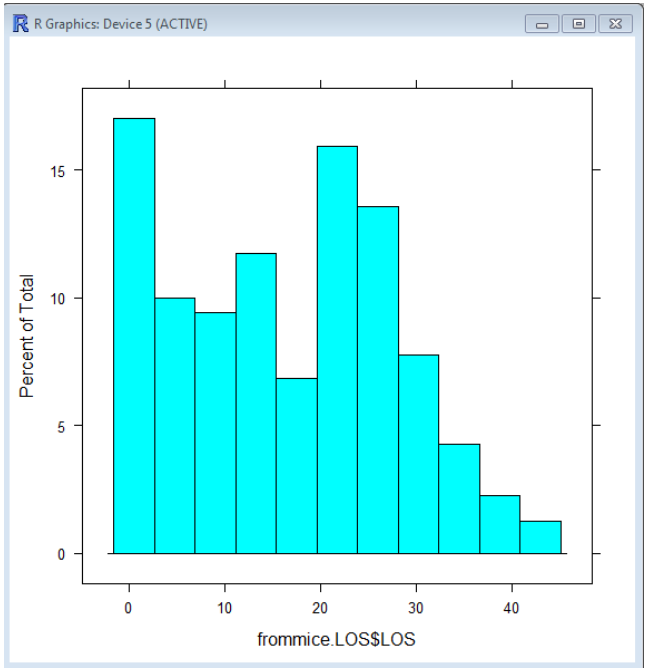

```
> histogram(frommice.LOS$LOS)
```



# Completely Random Imputation

A somewhat preferred alternative to unconditional mean imputation would be to select donor values completely at random. This can be interpreted as a single-cell hot-deck approach of sorts.

```
# good practice to assign a seed whenever there is a random
# component to imputation
> imp <- mice(tomice.LOS, method="sample", m=1, seed=29932)
> frommice.LOS <- complete(imp)

> histogram(frommice.LOS$LOS)
```



# Hot-Deck Imputation

The `hotdeck()` function within the `VIM()` package is an efficient way to conduct completely random imputation from within user-defined cells. You simply specific the categorical variable(s) defining a cell in the `domain_var=` argument as follows:

```
# there is not seed argument within the hotdeck function, but we
# can assign one up front
> set.seed(883447)
> fromHD.LOS <- hotdeck(data=sample,
                        variable="LOS",
                        domain_var=c("GENDER","Minority"))
> fromHD.LOS[1:10,]
```

| | GENDER | Minority | AGE | agecat | SUPERVISOR | weight_base | respond | Q1 | Q1_full | LOS | LOS_imp |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | F | N | 44.08214 | 0 | 0 | 13.281 | 0 | NA | NA | 27.581109 | TRUE |
| 2 | F | N | 40.91718 | 0 | 0 | 13.281 | 1 | 0 | 2 | 1.497604 | FALSE |
| 3 | F | N | 56.75017 | 0 | 0 | 13.281 | 0 | NA | NA | 22.329911 | TRUE |
| 4 | F | N | 41.66461 | 0 | 0 | 13.281 | 0 | NA | NA | 22.915811 | TRUE |
| 5 | M | N | 23.08282 | 1 | 0 | 13.281 | 0 | NA | NA | 32.246407 | TRUE |
| 6 | M | N | 63.91513 | 0 | 1 | 8.595 | 1 | 1 | 4 | 34.833676 | FALSE |
| 7 | M | N | 37.00205 | 1 | 0 | 13.281 | 1 | 1 | 5 | 3.082820 | FALSE |
| 8 | F | N | 57.49760 | 0 | 1 | 8.595 | 1 | 0 | 1 | 24.246407 | FALSE |
| 9 | F | N | 40.83504 | 0 | 0 | 13.281 | 0 | NA | NA | 2.168378 | TRUE |
| 10 | F | N | 57.58248 | 0 | 0 | 13.281 | 1 | 1 | 4 | 35.748118 | FALSE |

# Linear Regression Imputation

To request imputation from a linear regression model within the `mice()` function, specify `method="norm"` as follows:

```
> imp <- mice(tomice.LOS,method="norm",m=1,seed=29932)
> imp
```

```
Multiply imputed data set
Call:
mice(data = tomice.LOS, m = 1, method = "norm", seed = 29932)
Number of multiple imputations:  1
Missing cells per column:
    GENDER   Minority        AGE SUPERVISOR        LOS
         0          0          0          0        572
Imputation methods:
    GENDER   Minority        AGE SUPERVISOR        LOS
    "norm"     "norm"     "norm"     "norm"     "norm"
VisitSequence:
LOS
  5
PredictorMatrix:
           GENDER Minority AGE SUPERVISOR LOS
GENDER          0        0   0          0   0
Minority        0        0   0          0   0
AGE             0        0   0          0   0
SUPERVISOR      0        0   0          0   0
LOS             1        1   1          1   0
Random generator seed value:  29932
```

# Manipulating Predictor Variables

By default, the `mice()` function, along with most other comparable software, wants to use all available covariates on input data set as predictors. We can output, manipulate, and input the `predictorMatrix` object to customize things. For example, suppose we only wanted to use employee age and supervisory status to impute length of service. The syntax below accomplishes this task.

```
> newpred <- imp$predictorMatrix
> newpred["LOS",c("GENDER","Minority")] <- 0
> newpred
```

| | GENDER | Minority | AGE | SUPERVISOR | LOS |
|---|---|---|---|---|---|
| GENDER | 0 | 0 | 0 | 0 | 0 |
| Minority | 0 | 0 | 0 | 0 | 0 |
| AGE | 0 | 0 | 0 | 0 | 0 |
| SUPERVISOR | 0 | 0 | 0 | 0 | 0 |
| LOS | 0 | 0 | 1 | 1 | 0 |

```
> imp <- mice(tomice.LOS,
              method="norm",m=1,seed=29932,
              pred=newpred)
```

# Semi-Parametric Regression Imputation

Recall how one potential downside of any kind of explicit imputation model such as one based on linear regression. Indeed, we can see below how negative values of LOS were imputed for some missing cases. One work-around is to instead specify `method="pmm"` which invokes a semi-parametric approach called *predictive mean matching*.

```
> frommice.LOS <- complete(imp)
> frommice.LOS[1:5,]
```

| | GENDER | Minority | AGE | SUPERVISOR | LOS |
|---|---|---|---|---|---|
| 1 | F | N | 44.08214 | 0 | -0.4282047 |
| 2 | F | N | 40.91718 | 0 | 1.4976044 |
| 3 | F | N | 56.75017 | 0 | 21.1528104 |
| 4 | F | N | 41.66461 | 0 | 7.9577399 |
| 5 | M | N | 23.08282 | 0 | 3.7523102 |

```
# illustrating the PMM approach
> imp <- mice(tomice.LOS,method="pmm",m=1,seed=77232)
```

# Imputation of a Dichotomous Categorical Variable

When the outcome is dichotomous, or categorical with two levels, you can make use of the `method="logreg"` (logistic regression) as follows:

```
> tomice.Q1 <- sample[,
                      c("GENDER","Minority","AGE","SUPERVISOR","Q1")]
> imp <- mice(tomice.Q1,method="logreg",m=1,seed=63213)
```

If your variable with missing values is numeric, sometimes you get a message saying something like "`Type mismatch…Imputation method logreg is for categorical data. If you want that, turn variable…into a factor`." Code below does so:

```
> tomice.Q1$Q1 <- as.factor(tomice.Q1$Q1)
```

# Imputation of Categorical Variables with More than 2 Distinct Levels

Sometimes a categorical variable's codes fall on an ordinal scale. The Q1_full variable is one such example, where values from 1, 2, …, 5 represent varying degrees of agreement with a statement posed to the respondent. You can make use of `method="polr"` (proportional odds logistic regression) as follows:

```
> tomice.Q1_full <-
                    sample[,c("GENDER","Minority","AGE",
                              "SUPERVISOR","Q1_full")]
> imp <- mice(tomice.Q1_full,method="polr",m=1,seed=988234)
```

A more generalized version, also applicable to instances for nominal variables, where there is no inherent order amongst codes, is `method="polyreg"` (polytomous regression) as follows:

```
> imp <- mice(tomice.Q1_full,method="polyreg",m=1,seed=988234)
```

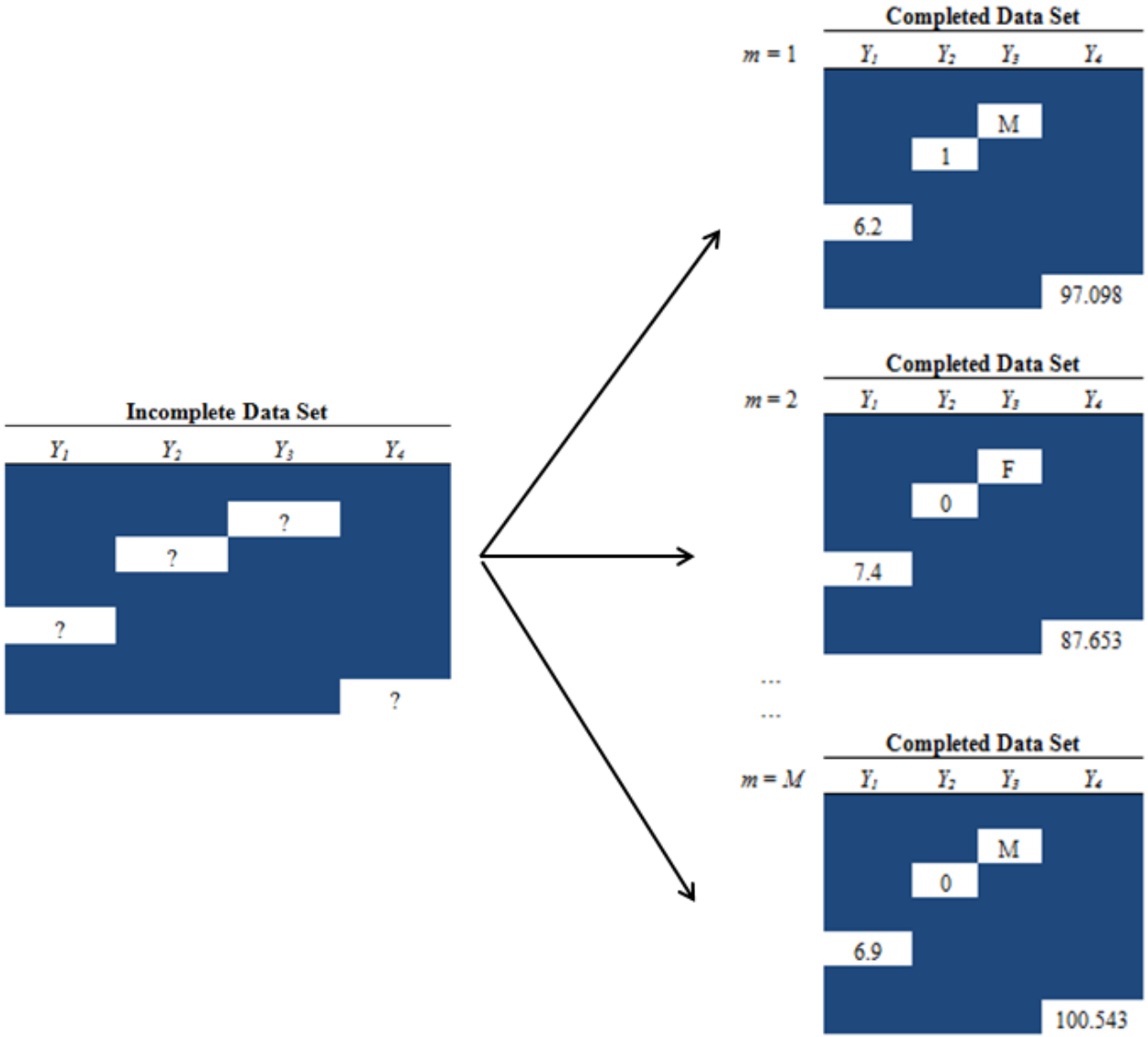
# Multiple Imputation as a Variance Inflation Method

Any kind of single imputation method will underestimate, or *attenuate*, the variance of a point estimate to some degree. Note that this is not an issue with weighting, only imputation. One of the most popular ways to properly inflate the variance to account for this is multiple imputation (MI) (Rubin, 1987).

The notion behind the method is to impute the missing values not once, but $M$ times ($M > 1$). The researcher has a choice for $M$, but a common assignment is $M = 5$.

The result is $M$ completed data sets, which are analyzed independently and the results combined in a straightforward way.

# Visualizing Multiple Imputation



# Rubin's Combination Rules – Univariate Inferences

Once we have created the set of *M* completed data sets, we analyze each independently to obtain *M* point estimates and *M* estimated variances.

Denote the $m^{th}$ completed data set point estimate $\hat{\theta}_m$

Denote the $m^{th}$ completed data set estimate variance $\text{var}\left(\hat{\theta}_m\right)$

Then the overall mean is $\hat{\theta} = \frac{1}{M}\sum_{m=1}^{M}\hat{\theta}_m$

And the overall variance is $\text{var}\left(\hat{\theta}\right) = \frac{1}{M}\sum_{m=1}^{M}\text{var}\left(\hat{\theta}_m\right) + \left(1+\frac{1}{M}\right)\frac{\sum_{m=1}^{M}\left(\hat{\theta}_m - \hat{\theta}\right)^2}{M-1}$

(within variance + between variance)

# Numerical Example of Univariate Inferences

For sake of an example, let us assume $M = 5$ and that we are interested in the sample mean of some variable $y$, and we compute the following estimated means and variances:

| $m$ | **Sample Mean** | **Estimated Variance of Sample Mean** |
|---|---|---|
| 1 | 111.2 | 4.4 |
| 2 | 115.3 | 3.2 |
| 3 | 112.6 | 4.5 |
| 4 | 111.5 | 4.7 |
| 5 | 102.3 | 5.3 |

Then the overall MI mean is $(111.2 + \ldots + 102.3)/5 = 110.58$

And the overall MI variance is $(4.4 + \ldots + 5.3)/5 +$

$(1 + 1/5)*(1/4)*[(111.2 - 110.58)^2 + \ldots (102.3 - 110.58)^2] = 33.2644$

# The Concept of “Proper” Imputation

One of the most important—yet often overlooked or not fully understood—concepts justifying the theory behind multiple imputation is that the plausible values must be derived from a *proper* procedure.

In essence, to be proper is to account for the uncertainty in the missing data model itself: even if we have the correct missingness assumption (e.g., that data are missing completely at random, or MCAR), we must account for the fact that a different realization of the missingness mechanism could have produced a somewhat different model and, therefore, different imputed values.

To help elucidate this distinction, we will consider several techniques to properly impute Q1, the dichotomous 0/1 indicator variable of a positive response in the employee satisfaction survey example.

# A Simple Missingness Assumption for Q1

Suppose we were willing to assume that the response process to the satisfaction survey was not influenced by any observed or unobserved factors. In other words, all *n* employees had approximately the same likelihood, or *propensity*, of participating, which is to say that Q1 is missing completely at random (MCAR).

Using only the *r* complete-cases in the data set sample, we find the following statistics:

1. The respondent mean: $\hat{\bar{y}}_r$
2. The estimated variance of the mean: $\text{var}(\hat{\bar{y}}) = \frac{s^2}{r} = \frac{\sum_{i \in r} \left( y_i - \hat{\bar{y}}_r \right)^2}{r(r-1)}$

Despite these values being unbiased under the MCAR assumption, for sake of discussion, let us suppose we sought to conduct multiple imputation to address the missingness…

# MI of Q1 Using an Implicit Model

It is instructive to see how a proper MI procedure operates in a single-cell, hot-deck model. Note that the procedure is *not* to simply impute missing values of Q1 by selecting *M* independent with-replacement samples of observed cases.

The proper method is to abide by the following two-step procedure described in Rubin and Schenker (1986) known as the *approximate Bayesian Bootstrap* (ABB):

1. Select a with-replacement sample from the observed cases of same size as the number of observed cases. Symbolize this sample set as *r**.
2. From *r**, select a with-replacement sample of size equaling the number of missing cases. Use these as imputed values for the missing cases.
3. Repeat this process independently *M* times to create *M* completed data sets.

# MI of Q1 Using an Explicit Model

While the ABB is a nonparametric approach based on an implicit model, there is a parametric analog based on an explicit model. In the single-cell, hot-deck setting we are currently considering, it proceeds as follows:

1. Denote the sample mean of the 0/1 indicator variable Q1 (i.e., an estimated proportion) as $\hat{p}$ and its variance $\mathrm{var}(\hat{p})$.
2. Simulate a draw of a "new" sample mean from its distribution. That is, draw a random normal variate, *z*, and then assign the new sample mean as $\hat{p}^* + z\sqrt{\mathrm{var}(\hat{p}^*)}$.
3. Draw $r_i$, a random uniform variate between 0 and 1. If $\hat{p}^* < r_i$, then assign Q1 as 1 for the $i^{\text{th}}$ respondent; otherwise assign Q1 as 0.
4. Repeat this process independently *M* times.

# The Ingenuity of "Proper" MI

So what's so special about "proper" MI in the single-cell, hot-deck setting?

It turns out that, in theory, as $M \rightarrow \infty$, the expected value of the overall MI variance approximates the estimated variance computed using the *r* observed cases. Hence, there is no longer an unjust variance reduction as in single imputation. This is true of other statistics as well, not just the sample mean.

But again, this is only guaranteed so long as the MI method is "proper."

# Assessing the Impact of MI: The Fraction of Missing Information

Aside from the obvious comparisons of the complete case point estimates versus those obtained from MI (as well as their associated variance estimates), there are a few informative ratios making use of the total, between, and within variance components worth noting.

The first is the *fraction of missing information* (FMI), which is defined as follows:

FMI = Between Variance / Total Variance

In an uninformative model such as a single-cell hot-deck, as $M \rightarrow \infty$, we can expect the FMI to approximate the nonresponse rate. To the extent that the FMI is less than the nonresponse rate, it is an indication of the predictive power of covariates exploited in the imputation model.

# Customizing the Predictor Matrix

As with most imputation software, the default strategy is to use all available variables to impute any and all missing values. As this is not always appropriate, you can customize the `mice()` package's predictor matrix by outputting, modifying, and supplying it in a subsequent step:

```
# initial run to output a predictor matrix to manipulate
> initial <- mice(sample,m=1)
> initial
```

PredictorMatrix:

| | GENDER | Minority | AGE | agecat | SUPERVISOR | weight_base | respond | Q1 | Q1_full | LOS |
|---|---|---|---|---|---|---|---|---|---|---|
| GENDER | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Minority | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| AGE | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| agecat | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| SUPERVISOR | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| weight_base | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| respond | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Q1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 1 |
| Q1_full | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| LOS | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 |

```
> newpred <- initial$predictorMatrix
# ensure certain columns are not used as predictors
> newpred[,c("weight_base","respond","agecat")] <- 0
```

# Customizing the Predictor Matrix (2)

There may also be occasions where we want to instruct R to ignore a variable (i.e., do not impute and do not use as a predictor). A general example of a variable of this type is some sort of unique record number. In the present case, we can "back fill" Q1 based on Q1_full, so we can ignore it for the imputation step. We do this in two steps:

```
# Step 1: set all column entries of predictor matrix to zero
> newpred[,"Q1"] <- 0

# Step 2: set the Q1 placeholder in the method vector to missing
> newmeth <- initial$meth
> newmeth["Q1"] <- ""
> newmeth
```

```
Q1      Q1_full          LOS
""   "polyreg"        "pmm"
```

# Customizing the Method Matrix

We saw in the previous slide how to assign a null value for the method associated with a particular variable subject to missingness, in effect "skipping" that variable. You can use a similar tactic to modify the default strategy.

For example, the default for any continuous variable such as LOS is `pmm`, a semi-parametric approach called *predictive mean matching*. Suppose we instead wanted a impute it via a more traditional linear regression model approach, `norm`.

The syntax below modifies the method matrix accordingly:

```
> newmeth["LOS"] <- "norm"
```

# Customized MI with *M* = 5

Now that we have customized our predictor matrix and method matrix, at last we run the `mice()` function to create *M* data sets. Assuming we felt *M* = 5 would be sufficient, the following syntax carries out the procedure:

```
> sample$Q1_full <- as.factor(sample$Q1_full)
> imp <- mice(sample,m=5,meth=newmeth,pred=newpred,seed=923884)
```

You can use the `complete()` function within the `mice()` package to extract any of the *M* data sets stored within the object `imp`:

```
# inspect the first five rows of first completed data set
> complete(imp,1)[1:5,]
```

| | GENDER | Minority | AGE | agecat | SUPERVISOR | weight_base | respond | Q1 | Q1_full | LOS |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | F | N | 44.08214 | 0 | 0 | 13.281 | 0 | NA | 5 | 19.615317 |
| 2 | F | N | 40.91718 | 0 | 0 | 13.281 | 1 | 0 | 2 | 1.497604 |
| 3 | F | N | 56.75017 | 0 | 0 | 13.281 | 0 | NA | 5 | 31.889788 |
| 4 | F | N | 41.66461 | 0 | 0 | 13.281 | 0 | NA | 5 | 27.380261 |
| 5 | M | N | 23.08282 | 1 | 0 | 13.281 | 0 | NA | 5 | 5.670677 |

# Stacking the *M* Data Sets

Another handy method for extracting the completed data sets is to specify "long" as the second argument of the `complete()` function. This will stack all *M* data sets on top of one another with a sequence variable `.imp` which can be used to partition them for subsequent analyses.

```
> sample.stacked <- complete(imp,"long")
> sample.stacked[1196:1205,]
```

| | .imp | .id | GENDER | Minority | AGE | agecat | SUPERVISOR | weight_base | respond | Q1 | Q1_full | LOS |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1196 | 1 | 1196 | F | Y | 40.66530 | 0 | 0 | 13.281 | 1 | 1 | 5 | 1.5824778 |
| 1197 | 1 | 1197 | F | Y | 43.66324 | 0 | 0 | 13.281 | 1 | 1 | 5 | 5.6646133 |
| 1198 | 1 | 1198 | M | N | 60.24641 | 0 | 1 | 8.595 | 1 | 0 | 2 | 1.0000000 |
| 1199 | 1 | 1199 | F | N | 33.83436 | 1 | 0 | 13.281 | 0 | NA | 4 | -10.3089117 |
| 1200 | 1 | 1200 | M | Y | 28.00000 | 1 | 0 | 13.281 | 1 | 1 | 4 | 7.0828200 |
| 1201 | 2 | 1 | F | N | 44.08214 | 0 | 0 | 13.281 | 0 | NA | 3 | 11.1723039 |
| 1202 | 2 | 2 | F | N | 40.91718 | 0 | 0 | 13.281 | 1 | 0 | 2 | 1.4976044 |
| 1203 | 2 | 3 | F | N | 56.75017 | 0 | 0 | 13.281 | 0 | NA | 5 | 27.6608874 |
| 1204 | 2 | 4 | F | N | 41.66461 | 0 | 0 | 13.281 | 0 | NA | 5 | 18.8855635 |
| 1205 | 2 | 5 | M | N | 23.08282 | 1 | 0 | 13.281 | 0 | NA | 2 | -0.8629163 |

# Back-Filling Q1 Based on Q1_full

Having all completed data sets stacked into one makes the task of back-filling in Q1 based on the newly imputed values of Q1_full:

```
> sample.stacked$Q1[sample.stacked$Q1_full=="5"] <- 1
> sample.stacked$Q1[sample.stacked$Q1_full=="4"] <- 1
> sample.stacked$Q1[sample.stacked$Q1_full=="3"] <- 0
> sample.stacked$Q1[sample.stacked$Q1_full=="2"] <- 0
> sample.stacked$Q1[sample.stacked$Q1_full=="1"] <- 0
> sample.stacked[1196:1205,]
```

| | .imp | .id | GENDER | Minority | AGE | agecat | SUPERVISOR | weight_base | respond | Q1 | Q1_full | LOS |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1196 | 1 | 1196 | F | Y | 40.66530 | 0 | 0 | 13.281 | 1 | 1 | 5 | 1.5824778 |
| 1197 | 1 | 1197 | F | Y | 43.66324 | 0 | 0 | 13.281 | 1 | 1 | 5 | 5.6646133 |
| 1198 | 1 | 1198 | M | N | 60.24641 | 0 | 1 | 8.595 | 1 | 0 | 2 | 1.0000000 |
| 1199 | 1 | 1199 | F | N | 33.83436 | 1 | 0 | 13.281 | 0 | 1 | 4 | -10.3089117 |
| 1200 | 1 | 1200 | M | Y | 28.00000 | 1 | 0 | 13.281 | 1 | 1 | 4 | 7.0828200 |
| 1201 | 2 | 1 | F | N | 44.08214 | 0 | 0 | 13.281 | 0 | 0 | 3 | 11.1723039 |
| 1202 | 2 | 2 | F | N | 40.91718 | 0 | 0 | 13.281 | 1 | 0 | 2 | 1.4976044 |
| 1203 | 2 | 3 | F | N | 56.75017 | 0 | 0 | 13.281 | 0 | 1 | 5 | 27.6608874 |
| 1204 | 2 | 4 | F | N | 41.66461 | 0 | 0 | 13.281 | 0 | 1 | 5 | 18.8855635 |
| 1205 | 2 | 5 | M | N | 23.08282 | 1 | 0 | 13.281 | 0 | 0 | 2 | -0.8629163 |

# Computing Rubin's Rules in R for a Scalar

For make inferences on a scalar (i.e., univariate inferences), we can consolidate all *M* point estimates and variances in a few different ways. The first is to do so "by hand":

```
> means <- c(111.2,115.3,112.6,111.5,102.3)
> variances <- c(4.4,3.2,4.5,4.7,5.3)

> mean(means)
[1] 110.58

> mean(variances) + (1+1/5)*var(means)
[1] 33.2644
```

# Computing Rubin's Rules in R for a Scalar (2)

The second approach is to use the `pool.scalar()` function within the `mice()` package. Note that in the output, "q" refers to quantity, "u" refers to uncertainty, "b" refers to between-variance, and "t" refers to total variance.

```
> pool.scalar(means,variances)

$m
[1] 5
$qhat
[1] 111.2 115.3 112.6 111.5 102.3
$u
[1] 4.4 3.2 4.5 4.7 5.3
$qbar
[1] 110.58
$ubar
[1] 4.42
$b
[1] 24.037
$t
[1] 33.2644
$fmi
[1] 0.8990752
```

# An Example with the Sample Data Set: Finding the Mean of the Q1 Indicator Variable

Suppose we were interested in the mean of Q1, which we can interpret as the proportion of people reacting positively to the question. Below is syntax to find the MI point estimate and variance:

```
# estimate the means
> point.ests <- aggregate(sample.stacked$Q1,list(sample.stacked$.imp),mean)
```

| | Group.1 | x |
|---|---|---|
| 1 | 1 | 0.7741667 |
| 2 | 2 | 0.7666667 |
| 3 | 3 | 0.7783333 |
| 4 | 4 | 0.7933333 |
| 5 | 5 | 0.7816667 |

```
# estimate the variances
> variances <- aggregate(sample.stacked$Q1,list(sample.stacked$.imp),var)
> variances <- variances$x/1200
```

| | Group.1 | x |
|---|---|---|
| 1 | 1 | 0.0001458154 |
| 2 | 2 | 0.0001491984 |
| 3 | 3 | 0.0001438954 |
| 4 | 4 | 0.0001367436 |
| 5 | 5 | 0.0001423385 |

# An Example with the Sample Data Set: Pooling the Estimates Together

```
> pool.scalar(point.ests$x,variances$x)

$m
[1] 5

$qhat
[1] 0.7741667 0.7666667 0.7783333 0.7933333 0.7816667

$u
[1] 0.0001458154 0.0001491984 0.0001438954 0.0001367436 0.0001423385

$qbar
[1] 0.7788333

$ubar
[1] 0.0001435983

$b
[1] 9.708333e-05

$t
[1] 0.0002600983

$fmi
[1] 0.4960604
```

# Comparing to the Complete Case Estimate

```
> mean.CC <- mean(sample$Q1, na.rm=TRUE)

> mean.CC
[1] 0.7834395


> var.CC <- var(sample$Q1, na.rm=TRUE) /
            length(subset(sample$Q1,sample$respond==1))
> var.CC

[1] 0.0002705934
```

# An Example with the Sample Data Set: Accounting for the Complex Survey Design

Recall that a stratified sample design was used, one involving variable selection probabilities. Technically, we should account for this during estimation:

```
# load Lumley's package
> library(survey)

# create a complex survey design object specifying features
> design <- svydesign(data=sample.stacked,
                      id=~0,              # code to say no clustering
                      strata=~SUPERVISOR,
                      weights=~weight_base)
> MI.stats <- svyby(~Q1, ~.imp, design, svymean)
> MI.stats
```

| | .imp | Q1 | se |
|---|---|---|---|
| 1 | 1 | 0.7797378 | 0.01199656 |
| 2 | 2 | 0.7733312 | 0.01210568 |
| 3 | 3 | 0.7847896 | 0.01187874 |
| 4 | 4 | 0.7997896 | 0.01155573 |
| 5 | 5 | 0.7889560 | 0.01177690 |

# An Example with the Sample Data Set: Accounting for the Complex Survey Design (2)

```
> pool.scalar(MI.stats$Q1,MI.stats$se^2)

$qhat
[1] 0.7797378 0.7733312 0.7847896 0.7997896 0.7889560

$u
[1] 0.0001439174 0.0001465475 0.0001411044 0.0001335349 0.0001386953

$qbar
[1] 0.7853208

$ubar
[1] 0.0001407599

$b
[1] 9.944086e-05

$t
[1] 0.0002600889

$fmi
[1] 0.5080101
```

# Multiple Imputation Competitors

Not every researcher is a fan of MI. Indeed, numerous alternative techniques have been proposed. Some are rooted in replication variance approaches:

- Efron (1994) → bootstrap re-imputation
- Rao and Shao (1992) → jackknife technique

While others are based on slightly modified paradigms:

- Fay (1996) and Kim and Fuller (2004) → fractional imputation

Still, there is little room to refute that MI is by far the most widely used method.